Lecture 05 - Interfaces and Generics

1

Interfaces

Other langs

```
interface Speaker {
    function speak();
}


class Dog implements Speaker {
    function speak() {
        // ...
    }
}

class Cat implements Speaker {
    function speak() {
        // ...
    }
}
```

Golang

```
type Dog struct {}

type Cat struct {}

func (Dog) Speak() string {
    return "woof"
}

func (Cat) Speak() string {
    return "meow"
}

type Elephant struct {

}

func (Elephant) Speak() string {
    return "puuuu"
}
```

let's create an interface

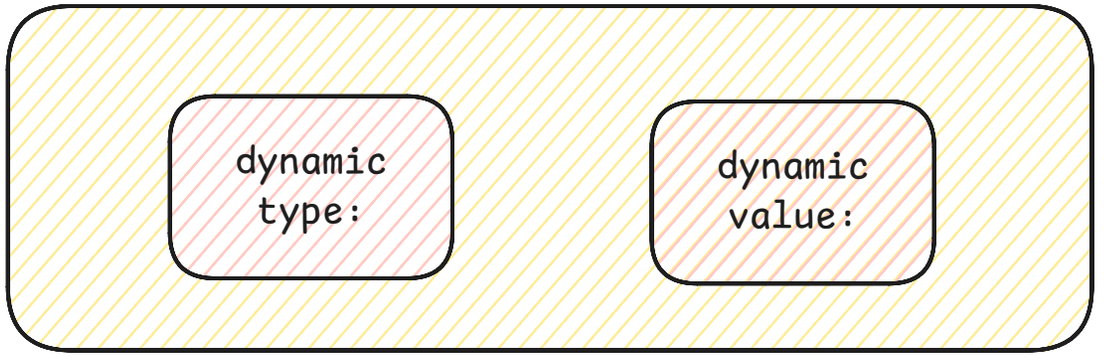
```
type Speaker interface {
    Speak() string
}

func MakeSound(s Speaker) {
    fmt.Println(s.Speak())
}

MakeSound(Dog{})
MakeSound(Cat{})

func MakeSoundDog(d Dog)
func MakeSoundCat(d Cat)
```

```
type FileStorage struct {}
type DatabaseStorage struct {}
type S3Storage struct {}
```

```
func (FileStorage) Save(...) {
    ...
}

func (DatabaseStorage) Save(...) {
    ...
}

func (S3Storage) Save(...) {
    ...
}
```

```
func ProcessFile(storage Storage) error {
    storage.Save()
}
```

interface



var s Speaker

s = Dog{}

static type: Speaker
dynamic type: Dog
dynamic value: Dog{}

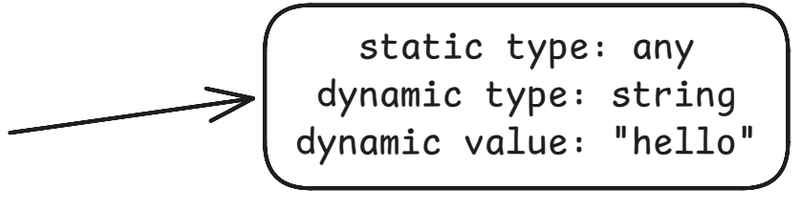
```
func Display(value interface{}) {

}
```

Go 1.18

```
func Display(value any) {

}
```

any

```
var x any = "hello"
```




```
var x any
```

0x...

dt

```
var x slice
```

```
type slice struct {
    arr *d
    len
    cap
}
```

```
var x any = "hello"

s := x.(string)

fmt.Println(s) // hello
```

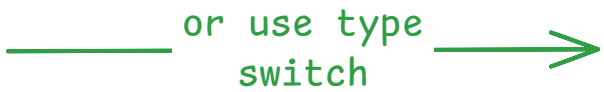
type assertion

```
var x any = 42

s := x.(string) // panic: interface{} is int, not string
```

Safe type assertion: comma-ok idiom

```
var x any = 42

s, ok := x.(string)

if ok {
    ...
}
```

```
var x any = 42

if s, ok := x.(string); ok {
    // process string
}

if s, ok := x.(int); ok {
    //
}
```




```
switch s := x.(type) {
case string:
    ...
case int:
    ...
default:
    ...
}
```